
Modül 11 — Kafka ile Mesajlaşma

Ders Notları

Spring Boot Edu



2026-09-23

İçindekiler

1. Öğrenme Hedefleri	2
2. Kavramlar	2
2.1 Topic, Partition ve Offset	2
2.2 Consumer Group'lar	2
2.3 Teslim Garantileri	3
3. Adım Adım Örnekler	3
3.1 Event Üretmek	4
3.2 Event Tüketmek	5
3.3 Hatalar, Retry'lar ve Dead Letter Topic	6
3.4 @RetryableTopic ile Bloklamayan Retry'lar	7
3.5 Kafka Transaction'ları	8
3.6 Transactional Outbox	9
3.7 Kafka Streams	11
3.8 Testcontainers ile Test	12
4. Sık Yapılan Hatalar ve En İyi Pratikler	13
5. Özet	13
6. İleri Okuma	14

1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- Topic, partition, offset ve consumer group kavramlarını açıklamak
- KafkaTemplate ile JSON event göndermek ve @KafkaListener ile almak
- Hataları yönetmek: retry'lar, dead letter topic (DLT) ve zehirli mesajlar
- @RetryableTopic ile bir partition'ı bloklamadan yeniden denemek
- Bir Kafka transaction'ı ile birden çok mesajı atomik yazmak
- Transactional outbox deseniyle bir veritabanı transaction'ından güvenilir event yayınlamak
- Küçük bir Kafka Streams topolojisi kurmak ve onu broker olmadan test etmek
- Kafka kodunu Testcontainers ile test etmek

Ön koşullar: Modül 06 (PostgreSQL, transaction'lar) · **Tahmini süre:** 6 saat · **Docker gerekir**

2. Kavramlar

2.1 Topic, Partition ve Offset

Kafka, dağıtık ve yalnızca sonuna eklenen bir **log**'dur. Producer'lar bir **topic**'e **kayıt** (key, value, header'lar) ekler. Bir topic **partition**'lara bölünür. Bir partition içinde sıra garantilidir ve her kaydın artan bir numarası, yani **offset**'i vardır.

```
topic "orders"  partition 0:  [0] [1] [2] [3] ...
                  partition 1:  [0] [1] ...
                  partition 2:  [0] [1] [2] ...
```

- **Key**, partition'ı belirler: Aynı key'e sahip kayıtlar her zaman aynı partition'a, sırayla düşer. Biz key olarak sipariş id'sini kullanıyoruz.
- Kayıtlar okununca **silinmez**. Saklama süresi dolana kadar kalırlar. Böylece yeni bir consumer geçmişini okuyabilir.

2.2 Consumer Group'lar

Bir **consumer group**, bir topic'in işini paylaşan consumer'lardan oluşur. Her partition'ı grubun tam olarak bir consumer'ı okur. Bu yüzden 3 partition'lı bir topic'i bir grubun en fazla 3 consumer'ı paralel işleyebilir. Her grup kendi offset'lerini hatırlar.

	“shipping” grubu	“invoicing” grubu
Alır	orders’taki her event’i	orders’taki her event’i o da
Paralellik	en fazla 3 örnek (3 partition)	en fazla 3 örnek

Farklı gruplar farklı uygulamalardır. Aynı uygulamanın örnekleri tek bir grubu paylaşır.

2.3 Teslim Garantileri

Varsayılan olarak bir consumer bir kaydı işler ve sonra offset’ini commit eder. Arada çökerse kayıt yeniden teslim edilir: **at-least-once** (en az bir kez). Bu yüzden consumer’lar **idempotent** olmalıdır: Aynı event’i iki kez işlemek, bir kez işlemekle aynı sonucu vermeli-dir (Ödev 1).

3. Adım Adım Örnekler

Docker açıkken uygulamayı başlatın. Kafka (KRaft modu, ZooKeeper yok) ve PostgreSQL kök dizindeki compose.yaml dosyasından başlar:

```
./mvnw -pl modules/11-messaging-kafka/lesson spring-boot:run
```

Kafka ayarları (spring: altında):

```
kafka:
  # bootstrap-servers defaults to localhost:9092 – the HOST listener of compose.yaml
  producer:
    key-serializer: org.apache.kafka.common.serialization.StringSerializer
    value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
    acks: all # the broker confirms only after the write
    ↪ is replicated
  consumer:
    auto-offset-reset: earliest # a new group starts at the beginning of
    ↪ the topic
    key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
    # ErrorHandlingDeserializer: a message that is not valid JSON goes to the error
    ↪ handler (→ DLT)
    # instead of failing the consumer forever
    value-deserializer:
    ↪ org.springframework.kafka.support.serializer.JsonDeserializer
    properties:
      spring.deserializer.value.delegate.class:
    ↪ org.springframework.kafka.support.serializer.JsonDeserializer
```

```
spring.json.trusted.packages: com.springbootedu.messagingkafka.*
streams:
  application-id: bookstore-sales-counter    # also the consumer group of the Streams
↪ application
```

i Not

Spring Boot 4.1'de Kafka için Docker Compose service connection yoktur. Burada gerek de yoktur: `spring.kafka.bootstrap-servers` varsayılan olarak `localhost:9092`'dir. Bu, `compose.yaml` dosyasının bilgisayarınızdaki uygulamalar için açtığı listener'dır.

3.1 Event Üretmek

Event bir record'dur ve `JacksonJsonSerializer` ile JSON'a çevrilir:

```
@Component
public class OrderEvents {

    public static final String TOPIC = "orders";
    public static final String DEAD_LETTER_TOPIC = "orders.DLT";

    private final KafkaTemplate<String, OrderPlaced> kafka;

    public OrderEvents(KafkaTemplate<String, OrderPlaced> kafka) {
        this.kafka = kafka;
    }

    public CompletableFuture<SendResult<String, OrderPlaced>> publish(OrderPlaced order)
    ↪ {
        return kafka.send(TOPIC, order.orderId(), order);           // asynchronous:
        ↪ returns at once
    }
}
```

- `send` asenkron dur. Dönen `CompletableFuture`, broker yazmayı onayladığında tamamlanır. `acks: all` ile bu, tüm in-sync replikalar kaydı aldıktan sonradır.
- Serializer ayrıca Java tipini taşıyan bir `__TypeId__` header'ı yazar. Consumer bunu kullanır (bkz. `spring.json.trusted.packages`).

Topic'ler bean olarak tanımlanır. Boot'un `KafkaAdmin`'i onları açılışta oluşturur:

```
@Configuration(proxyBeanMethods = false)
public class TopicConfiguration {
```

```

@Bean
NewTopic ordersTopic() {
    return TopicBuilder.name(OrderEvents.TOPIC).partitions(3).replicas(1).build();
    ↪ // 3 = max. parallel consumers
}

@Bean
NewTopic ordersDeadLetterTopic() {
    return TopicBuilder.name(OrderEvents.DEAD_LETTER_TOPIC).partitions(1).replicas(1).build();
}

```

== 3.1 Produce
sent to orders-1 at offset 0

3.2 Event Tüketmek

```

@Component
class ShippingListener {

    private static final Logger log = LoggerFactory.getLogger(ShippingListener.class);

    private final Shipments shipments;

    ShippingListener(Shipments shipments) {
        this.shipments = shipments;
    }

    @KafkaListener(topics = OrderEvents.TOPIC, groupId = "shipping")
    void onOrderPlaced(OrderPlaced order) { // JSON → record
        ↪ by the deserializer
        if (order.quantity() <= 0) {
            throw new InvalidOrderException("quantity must be positive, was " +
                ↪ order.quantity());
        }
        shipments.ship(order);
        log.info("Shipping {} × {} for order {}", order.quantity(), order.isbn(),
        ↪ order.orderId());
    }
}

```

- @KafkaListener, topic'i yoklayan (poll) ve her kayıt için metodu çağıran bir listener container başlatır.
- Ayarlardaki ErrorHandlerDeserializer, metot çağrılmadan önce JSON'ı bir OrderPlaced'e çevirir.

- InvoiceListener (bölüm 3.4) aynı topic'i invoicing grubunda dinler. İki grup da her event'i alır.

Gruplara ve **lag**'lerine (ne kadar geride olduklarına) bakın:

```
docker compose exec kafka /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 --describe --group shipping
```

GROUP	TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG
shipping	orders	0	3	3	0
shipping	orders	1	3	3	0
shipping	orders	2	2	2	0

3.3 Hatalar, Retry'lar ve Dead Letter Topic

Bir listener exception fırlattığında ne olacağına Spring Kafka'nın error handler'ı karar verir. Önlem alınmazsa hep başarısız olan tek bir kayıt sonsuza kadar yeniden denenir ve arkasındaki partition takılı kalır. Bizim handler'ımız iki kez yeniden dener, sonra kaydı **dead letter topic** orders.DLT'ye park eder:

```
@Configuration(proxyBeanMethods = false)
public class ErrorHandlingConfiguration {

    @Bean // Boot adds a CommonErrorHandler
    ↪ bean to every listener
    DefaultErrorHandler kafkaErrorHandler(ProducerFactory<Object, Object> producers) {
        var recoverer = new DeadLetterPublishingRecoverer(deadLetterTemplate(producers),
            (record, exception) -> new TopicPartition(OrderEvents.DEAD_LETTER_TOPIC,
                ↪ -1)); // -1: any partition
        var handler = new DefaultErrorHandler(recoverer, new FixedBackOff(200, 2)); // 1
        ↪ try + 2 retries, 200 ms apart
        handler.addNotRetryableExceptions(InvalidOrderException.class); //
    ↪ hopeless: straight to the DLT
        return handler;
    }
}
```

- InvalidOrderException yeniden denenmez: Miktarı 0 olan bir sipariş asla geçerli olmayacaktır, bu yüzden hemen DLT'ye gider.
- Recoverer, kafka_dlt-exception-message ve kafka_dlt-original-topic gibi header'lar ekler. Böylece kayıt daha sonra incelenip yeniden oynatılabilir.
- **Zehirli mesaj** (poison pill), deserialize bile edilemeyen bir kayıttır (ör. this is not json). ErrorHandlingDeserializer hatayı yakalar ve kaydı error handler'a verir. Handler **orijinal byte'ları** DLT'ye yeniden yayımlar. O olmasaydı consumer aynı kayıta sonsuz bir döngüde hata verirdi.

DeadLetterTest iki durumu da kontrol eder.

3.4 @RetryableTopic ile Bloklamayan Retry'lar

Yerinde yeniden denemek, beklerken partition'ı bloklar. Arkadaki bir servis bir süre kapalıysa başarısız kaydı yoldan çekip daha sonra yeniden denemek daha iyidir:

```
@Component
class InvoiceListener {

    private static final Logger log = LoggerFactory.getLogger(InvoiceListener.class);

    private final Invoices invoices;

    InvoiceListener(Invoices invoices) {
        this.invoices = invoices;
    }

    @RetryableTopic(attempts = "3", // 1 try + 2 retries
                   backOff = @BackOff(delay = 500, multiplier = 2), // 500 ms, then 1 s
                   retryTopicSuffix = "-invoicing-retry", //
↪ orders-invoicing-retry-500, -1000
                   dltTopicSuffix = "-invoicing-dlt", //
↪ orders-invoicing-dlt
                   sameIntervalTopicReuseStrategy =
↪ SameIntervalTopicReuseStrategy.MULTIPLE_TOPICS)
    @KafkaListener(topics = OrderEvents.TOPIC, groupId = "invoicing")
    void onOrderPlaced(OrderPlaced order) {
        int attempt = invoices.countAttempt(order.orderId());
        if (order.customerId().startsWith("flaky-") && attempt < 3) {
            throw new IllegalStateException("invoicing service unavailable (attempt " +
↪ attempt + ")");
        }
        invoices.invoice(order.orderId());
        log.info("Invoiced order {} after {} attempt(s)", order.orderId(), attempt);
    }

    @DltHandler
    void onGiveUp(OrderPlaced order) {
        log.warn("Invoicing gave up on order {}", order.orderId());
    }
}
```

Spring Kafka retry topic'lerini ve DLT'yi kendisi oluşturur ve her birinde bir listener çalıştırır:

```
orders —hata→ orders-invoicing-retry-500 —hata→ orders-invoicing-retry-1000
↪ —hata→ orders-invoicing-dlt
```

Topic adları milisaniye cinsinden bekleme süresiyle biter. Başarısız bir kayıt beklerken ana topic akmaya devam eder. Bedeli şudur: Aynı siparişin event'leri artık **sırasız** işlenebilir.

```
== 3.2 Consume (group shipping, group invoicing)
flaky order invoiced after 3 attempts
```

3.5 Kafka Transaction'ları

Depolar arasında stok taşımak iki mesaj üretir: bir çekme ve bir yatırma. Bir consumer asla yalnızca birini görmemelidir. Bir Kafka transaction'ı birden çok yazmayı atomik yapar:

```
@Component
public class StockTransfers {

    public static final String TOPIC = "stock-transfers";

    private final DefaultKafkaProducerFactory<String, String> transactionalProducers;
    private final KafkaTemplate<String, String> kafka;

    public StockTransfers(ProducerFactory<Object, Object> producers) {
        this.transactionalProducers = new
            ↪ DefaultKafkaProducerFactory<>(producers.getConfiguratioProperties(),
                new StringSerializer(), new StringSerializer());
        transactionalProducers.setTransactionIdPrefix("stock-transfer-"); // makes the
        ↪ producers transactional
        this.kafka = new KafkaTemplate<>(transactionalProducers);
    }

    public void transfer(String transferId, String from, String to, int quantity, boolean
        ↪ failInTheMiddle) {
        kafka.executeInTransaction(operations -> {
            operations.send(TOPIC, transferId, "withdraw " + quantity + " from " + from);
            operations.flush(); // really written to
            ↪ the log ...
            if (failInTheMiddle) {
                throw new IllegalStateException("failure after the first message"); // ...
                ↪ and then aborted
            }
            operations.send(TOPIC, transferId, "deposit " + quantity + " to " + to);
            return null;
        });
    }
}
```

- setTransactionIdPrefix producer'ı transactional yapar. executeInTransaction, call-back dönünce commit eder, exception fırlatınca abort eder.

- Abort edilen bir mesaj log’da zaten olabilir. `isolation.level=read_committed` olan consumer’lar onu atlar. `read_uncommitted` consumer’lar (Kafka client’ının varsayılanı) onu yine görür. `KafkaTransactionTest` ikisini de gösterir.

! Önemli

Bir Kafka transaction’ı **yalnızca Kafka’yı** kapsar. Bir PostgreSQL yazmasını içeremez. “Veritabanına kaydet ve event gönder” için outbox gerekir (sonraki bölüm).

3.6 Transactional Outbox

“Siparişi kaydet, sonra event’i gönder” yaklaşımında iki hata penceresi vardır. Uygulama commit’ten sonra ama göndermeden önce çökerse event kaybolur. Önce gönderir ve sonra commit başarısız olursa event var olmayan bir siparişi anlatır. **Outbox deseni**, event’i aynı veritabanı transaction’ına koyar:

```
CREATE TABLE outbox (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  -- also the sending
  ⇨ order
  aggregate_id VARCHAR(36) NOT NULL,                             -- becomes the Kafka key
  event_type   VARCHAR(100) NOT NULL,
  payload      TEXT NOT NULL,                                     -- the event as JSON
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  sent_at      TIMESTAMPTZ                                         -- NULL = not yet sent
);

CREATE INDEX outbox_unsent ON outbox (id) WHERE sent_at IS NULL;  -- the relay only
⇨ reads unsent rows
```

```
@Service
public class OrderService {

  private final JdbcClient jdbc;
  private final JsonMapper json;

  public OrderService(JdbcClient jdbc, JsonMapper json) {
    this.jdbc = jdbc;
    this.json = json;
  }

  @Transactional
  public OrderPlaced place(String customerId, String isbn, int quantity) {
    OrderPlaced event = new OrderPlaced(UUID.randomUUID().toString(), customerId,
    ⇨ isbn, quantity);
```

```

        jdbc.sql("INSERT INTO orders (id, customer_id, isbn, quantity) VALUES (:id,
↪ :customer, :isbn, :quantity)")
            .param("id", event.orderId()).param("customer", customerId)
            .param("isbn", isbn).param("quantity", quantity)
            .update();
        jdbc.sql("INSERT INTO outbox (aggregate_id, event_type, payload) VALUES (:id,
↪ :type, :payload)")
            .param("id", event.orderId())
            .param("type", OrderPlaced.class.getSimpleName())
            .param("payload", json.writeValueAsString(event))
            .update();
        reserveStock(isbn, quantity); // may fail → the order AND the
↪ event are rolled back
        return event;
    }

```

Stok kontrolü başarısız olursa sipariş **ve** outbox satırı birlikte geri alınır. Bir relay satırları daha sonra gönderir:

```

@Component
public class OutboxRelay {

    record PendingEvent(long id, String payload) {
    }

    private final JdbcClient jdbc;
    private final JsonMapper json;
    private final OrderEvents events;

    public OutboxRelay(JdbcClient jdbc, JsonMapper json, OrderEvents events) {
        this.jdbc = jdbc;
        this.json = json;
        this.events = events;
    }

    @Scheduled(fixedDelayString = "${bookstore.outbox.relay-interval}")
    @Transactional
    public int relay() throws ExecutionException, InterruptedException, TimeoutException
    ↪ {
        List<PendingEvent> pending = jdbc.sql("""
            SELECT id, payload FROM outbox
            WHERE sent_at IS NULL
            ORDER BY id
            LIMIT 100
            FOR UPDATE SKIP LOCKED""") // several relays
↪ never send the same row
        .query(PendingEvent.class)
    }

```

```

        .list();
    for (PendingEvent event : pending) {
        events.publish(json.readValue(event.payload(), OrderPlaced.class))
            .get(10, TimeUnit.SECONDS); // wait for the
        ↪ broker's acknowledgement
        jdbc.sql("UPDATE outbox SET sent_at = now() WHERE id = :id").param("id",
    ↪ event.id()).update();
    }
    return pending.size();
}
}

```

- FOR UPDATE SKIP LOCKED, birkaç uygulama örneğinin relay'i aynı anda, bir satırı iki kez göndermeden çalıştırmasını sağlar.
- Relay, bir satırı gönderildi olarak işaretlemekten önce broker'ın onayını bekler. Arada çökerse satır yeniden başladıktan sonra tekrar gönderilir: at-least-once, bu yüzden consumer'lar idempotent olmalıdır.
- Debezium gibi araçlar outbox tablosunu yoklamak yerine veritabanının log'undan okur. Desen aynı kalır.

3.7 Kafka Streams

Kafka Streams, topic'leri uygulamanızın içinde sürekli işlemek için bir kütüphanedir. Bu topoloji kitap başına satılan kopya sayısını tutar:

```

@Configuration(proxyBeanMethods = false)
@EnableKafkaStreams
public class SalesCounter {

    public static final String STORE = "sales-per-book";
    public static final String OUTPUT_TOPIC = "sales-per-book";

    @Bean
    public KStream<String, OrderPlaced> salesPerBook(StreamsBuilder builder) {
        var orderSerde = new JacksonJsonSerde<>(OrderPlaced.class).ignoreTypeHeaders();

        KStream<String, OrderPlaced> orders =
            builder.stream(OrderEvents.TOPIC, Consumed.with(Serdes.String(),
    ↪ orderSerde));
        orders.filter((orderId, order) -> order != null && order.quantity() > 0)
            .groupBy((orderId, order) -> order.isbn(), Grouped.with(Serdes.String(),
    ↪ orderSerde)) // re-key
            .aggregate(() -> 0L, (isbn, order, total) -> total + order.quantity(),
                Materialized.<String, Long, KeyValueStore<Bytes, byte[]>>as(STORE)
    ↪ // local state

```

```

        .withKeySerde(Serdes.String()).withValueSerde(Serdes.Long()))
        .toStream()
        .to(OUTPUT_TOPIC, Produced.with(Serdes.String(), Serdes.Long()));
        ↪ // every update
    return orders;
}
}

```

- `groupBy`, key'i sipariş id'sinden ISBN'e değiştirir. Kafka Streams bunun için kayıtları dahili bir repartition topic'inden geçirir.
- `aggregate`, ara toplamaları yerel bir **state store**'da tutar. Store bir changelog topic'i ile yedeklenir. Yeniden başlatmadan sonra toplamalar oradan geri yüklenir ve sayım kaldığı yerden devam eder.
- `@EnableKafkaStreams` ve `spring.kafka.streams.application-id`, topolojiyi uygulamayla birlikte başlatır.

`SalesCounterTopologyTest`, topolojiyi `TopologyTestDriver` ile test eder: broker yok, Docker yok, test başına milisaniyeler.

! Uyarı

Yerel state, `spring.kafka.streams.state-dir` içinde durur (varsayılan olarak geçici dizin altında). Uygulamayı yeni ve boş bir Kafka'ya yönlendirirseniz bu dizini silin: Eski yerel state yeni broker'la uyuşmaz.

3.8 Testcontainers ile Test

```

@TestConfiguration(proxyBeanMethods = false)
public class TestcontainersConfiguration {

    static final KafkaContainer KAFKA = new KafkaContainer("apache/kafka:4.2.1");
    ↪ // KRaft, no ZooKeeper

    static final PostgreSQLContainer POSTGRES = new PostgreSQLContainer(
        DockerImageName.parse("pgvector/pgvector:0.8.6-
    ↪ pg18").asCompatibleSubstituteFor("postgres"));

    @Bean
    @ServiceConnection                                // sets
    ↪ spring.kafka.bootstrap-servers
    KafkaContainer kafkaContainer() {
        return KAFKA;
    }

    @Bean

```

```
@ServiceConnection
PostgreSQLContainer postgresContainer() {
    return POSTGRES;
}
```

- `org.testcontainers.kafka.KafkaContainer`, resmî `apache/kafka` image'ini KRaft modunda çalıştırır. `@ServiceConnection` bootstrap sunucularını ayarlar.
- Mesajlaşma asenkronudur: Testler bir event gönderir ve sonra etkisi görünene kadar `Awaitility` ile bekler (`await().atMost(...)`). Asla sabit bir `Thread.sleep` kullanmayın.
- Her test yeni ve rastgele sipariş id'leri kullanır. Böylece testler ortak topic'lerde birbirinin event'lerini görmez.

4. Sık Yapılan Hatalar ve En İyi Pratikler

!! Dikkat

Bir veritabanı transaction'ı içinde Kafka mesajı gönderip ikisinin birlikte gerçekleştiğini varsaymayın. Outbox desenini kullanın veya aradaki boşluğu bilerek kabul edip yönetin.

- **Yapın:** Key'i ihtiyacınız olan sıraya göre seçin. Bir siparişin event'leri aynı key'i paylaşmalıdır.
- **Yapmayın:** Bir grupta topic'in partition sayısından fazla consumer oluşturmayın. Fazla consumer'lar boşa kalır.
- **Yapın:** Consumer'ları idempotent yapın. At-least-once teslim, tekrarların olacağı anlamına gelir.
- **Yapmayın:** Tek bir kötü kaydın bir partition'ı bloklamasına izin vermeyin. Dead letter topic'li bir error handler yapılandırın ve bir `ErrorHandlingDeserializer` kullanın.
- **Yapın:** `spring.json.trusted.packages` değerine yalnızca kendi paketlerinizi yazın. Header'lardaki rastgele tiplere deserialize etmek bir güvenlik riskidir.
- **Yapmayın:** Partition'lar arasında veya bloklamayan retry'lardan sonra sıra bekleme-yin.

5. Özet

- Kafka event'leri partition'lara bölünmüş, sıralı log'larda saklar. Key partition'ı, dola-yısıyla sırayı belirler.
- Her consumer group her event'i bir kez alır. Bir grup içinde partition'lar örnekler arasında paylaşılır.
- `DefaultErrorHandler` yeniden dener, sonra umutsuz kayıtları ve zehirli mesajları bir dead letter topic'e gönderir.

- `@RetryableTopic`, ana topic'i bloklamadan ayrı topic'ler üzerinden yeniden dener.
- Kafka transaction'ları, `read_committed` consumer'lar için birden çok yazmayı atomik yapar, ancak yalnızca Kafka içinde.
- Transactional outbox event'i veriyle birlikte kaydeder, bir relay onu en az bir kez yayınlar.
- Kafka Streams topic'leri yerel ve hataya dayanıklı state ile sürekli işler. `Topology-TestDriver` onu broker olmadan test eder.

Sıradaki adım: [Ödevler](#)

6. İleri Okuma

- [Spring for Apache Kafka Reference](#)
- [Spring Kafka — Handling Exceptions · Non-Blocking Retries · Transactions](#)
- [Spring Kafka — Kafka Streams Support](#)
- [Spring Boot — Apache Kafka Support](#)
- [Apache Kafka Documentation](#)
- [Pattern: Transactional Outbox](#)